

Design Patterns

Command.....	2
Data Transfer Object (DTO).....	3
Adapter	4
Strategy	5
Façade	6
Event Sourcing	7
Publish-Subscribe (aka: Observer)	8
Proxy	9
Context Object	10
Reflection	11
Chain of Responsibility.....	12
Composite	13
Interpreter	14
State	15
Template Method	16
Decorator	17
Domain Model	18
Domain Object	19
Layers	20
Model-View-Controller	21
Pipes and Filters	22
Shared Repository	23
Microkernel.....	24
Abstract Factory Class.....	25
Mediator	26

Command

(4-BSc_SWA_Conditions_Significance-Solution)

Problem. Method calls must be issued, delayed, queued, or undone independently of their sender.

Context. Systems needing task scheduling, macro recording, or undo/redo.

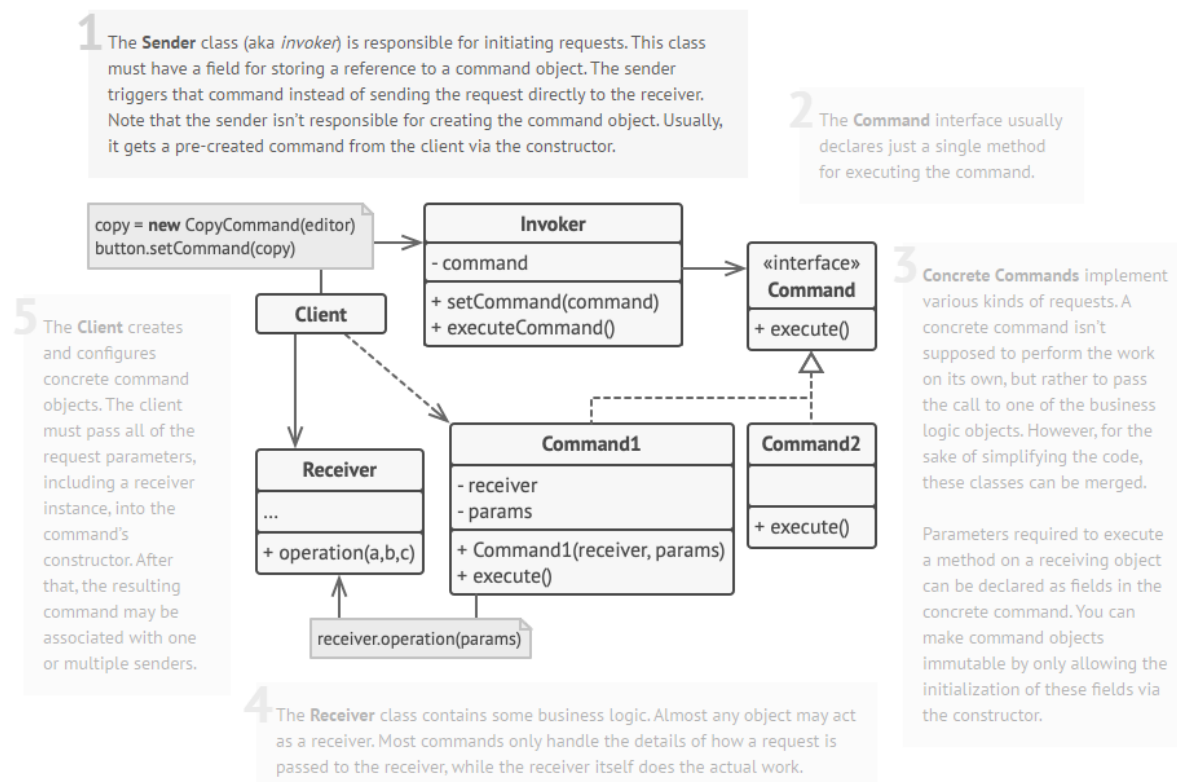
Forces. Decoupling requester from executor.

Solution. Encapsulate an action and its receiver in a `Command` object with a common `execute()` interface.

Consequences. Enables queuing, undo, and flexible invocation; adds extra objects.

Related. Strategy, Memento, Composite.

Uses. GUI buttons, transactional job queues.



Data Transfer Object (DTO)

(4-BSc_SWA_Conditions_Significance-Solution)

Problem. Transferring many fine-grained data calls across layers causes latency.

Context. Distributed or layered systems.

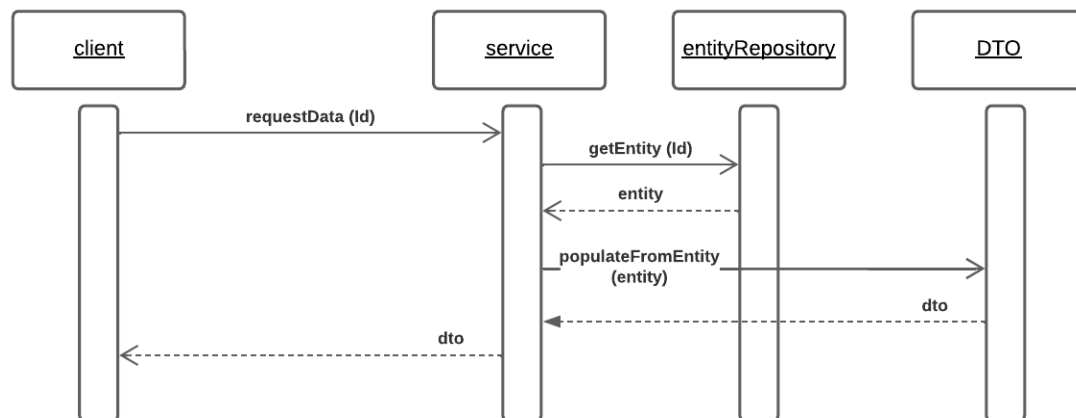
Forces. Minimize remote invocations.

Solution. Aggregate related data into a serializable object passed in one call.

Consequences. Reduces network overhead but duplicates data structures.

Related. Value Object, Facade.

Uses. REST/GraphQL payloads, EJBs.



Adapter

(4-BSc_SWA_Conditions_Significance-Solution)

Problem. Two components must collaborate despite incompatible interfaces.

Context. Integrating legacy or third-party APIs.

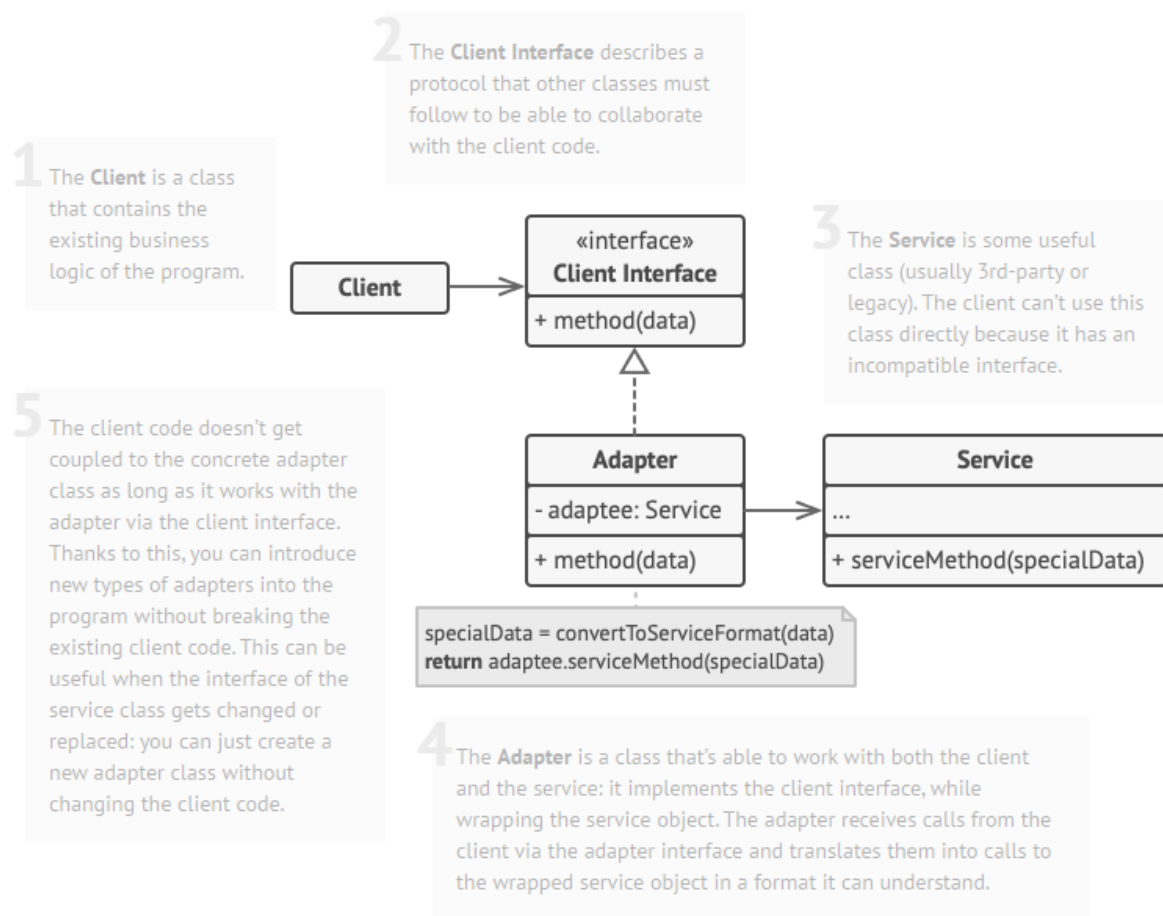
Forces. Reuse existing code without modification.

Solution. Wrap the adaptee in an adapter that implements the target interface and delegates calls.

Consequences. Enables reuse; adds one indirection layer.

Related. Bridge, Decorator, Proxy.

Uses. JDBC drivers, interface converters.



Strategy

(4-BSc_SWA_Conditions_Significance-Solution, 6-BSc_SWA_Method_DomainDrivenDesign-Solution)

Problem. Multiple algorithms exist for one task and must vary independently.

Context. Behavioural variation at runtime.

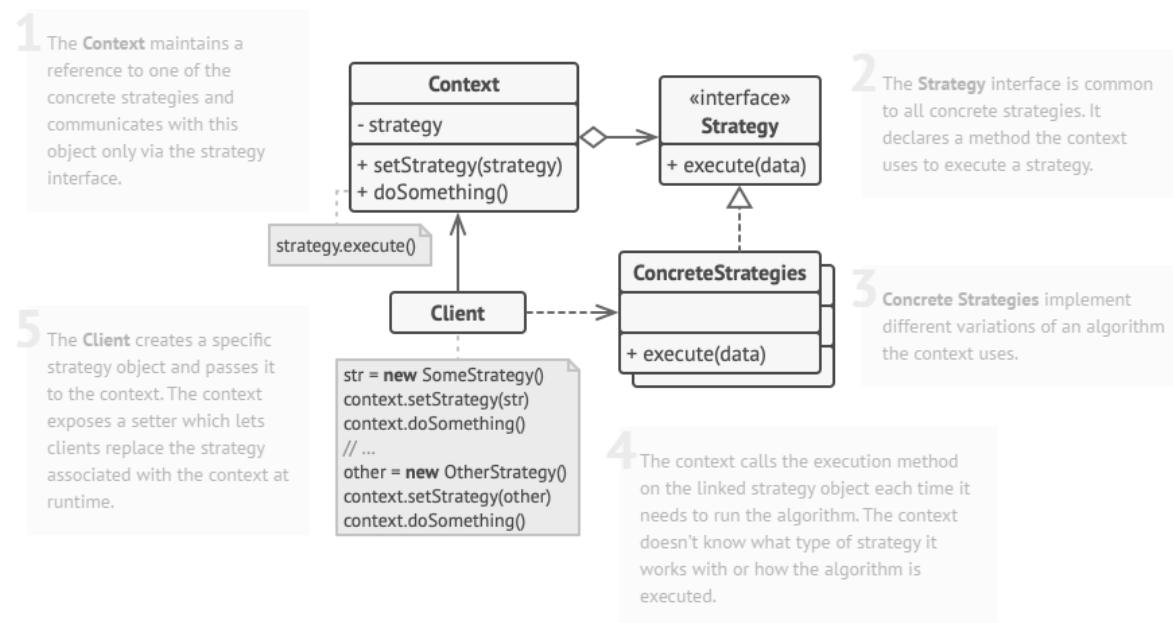
Forces. Avoid conditional logic and promote extensibility.

Solution. Define a family of interchangeable strategies implementing a common interface; context delegates execution.

Consequences. Adds flexibility; increases number of classes.

Related. State, Template Method.

Uses. Sorting, payment or compression algorithms.



Façade

(4-BSc_SWA_Conditions_Significance-Solution)

Problem. Complex subsystems overwhelm clients with many classes.

Context. Large frameworks or APIs.

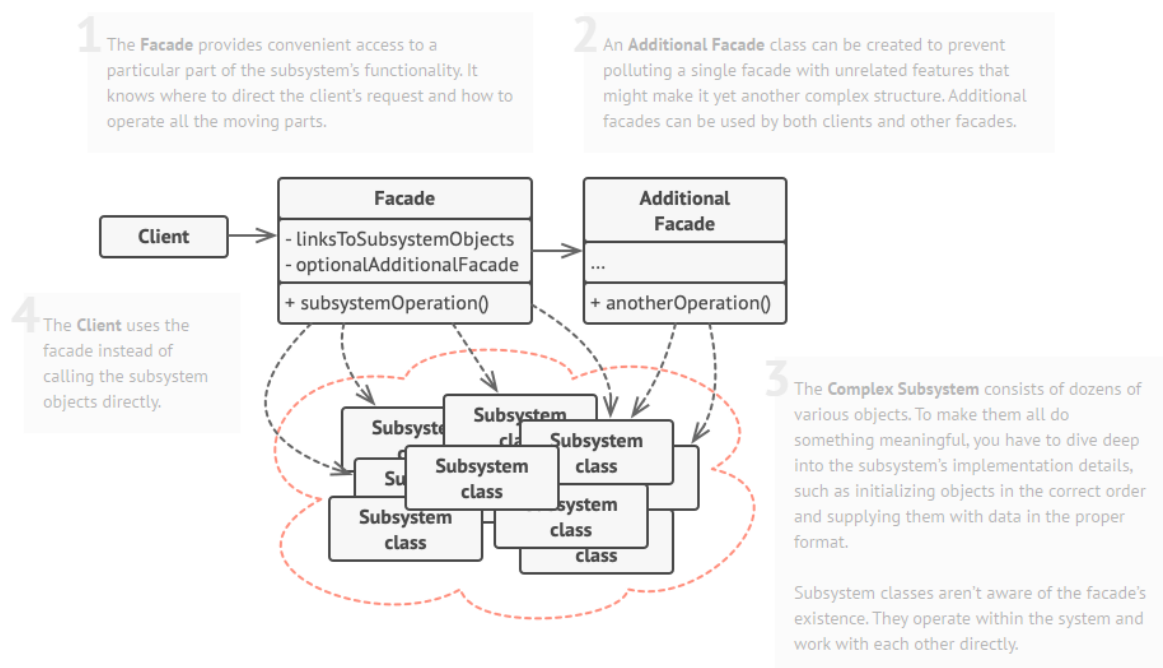
Forces. Reduce coupling and simplify use.

Solution. Provide a single high-level interface that wraps and coordinates subsystem components.

Consequences. Simplifies usage; may hide advanced features.

Related. Adapter, Mediator, Layers.

Uses. API gateways, persistence or logging libraries.



Event Sourcing

(4-BSc_SWA_Conditions_Significance-Solution)

Problem. Current-state persistence loses audit and history information.

Context. Systems requiring traceability or temporal reconstruction.

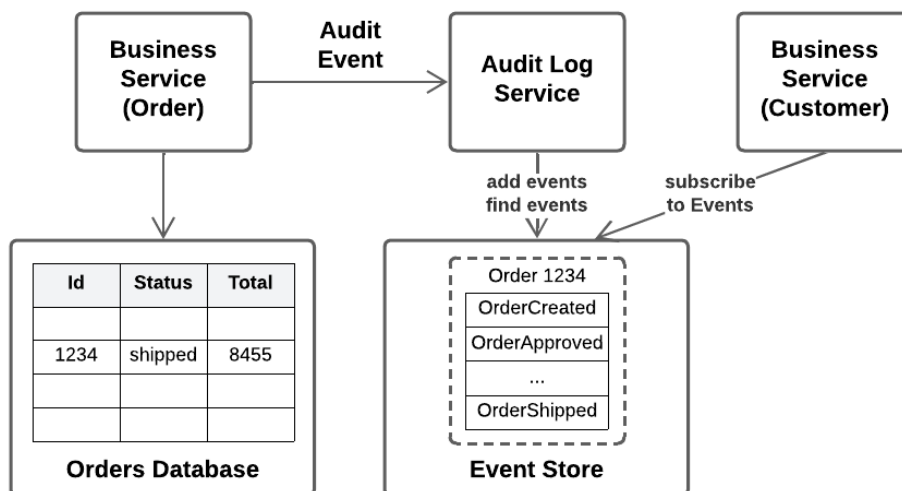
Forces. Balance auditability and performance.

Solution. Store every state-changing event and rebuild state by replaying them.

Consequences. Provides full audit trail; complicates queries.

Related. CQRS, Observer.

Uses. Banking ledgers, Git, Kafka event stores.



Publish-Subscribe (aka: Observer)

(4-BSc_SWA_Conditions_Significance-Solution)

Problem. Many objects need automatic updates when another changes.

Context. Event-driven GUIs or distributed notifications.

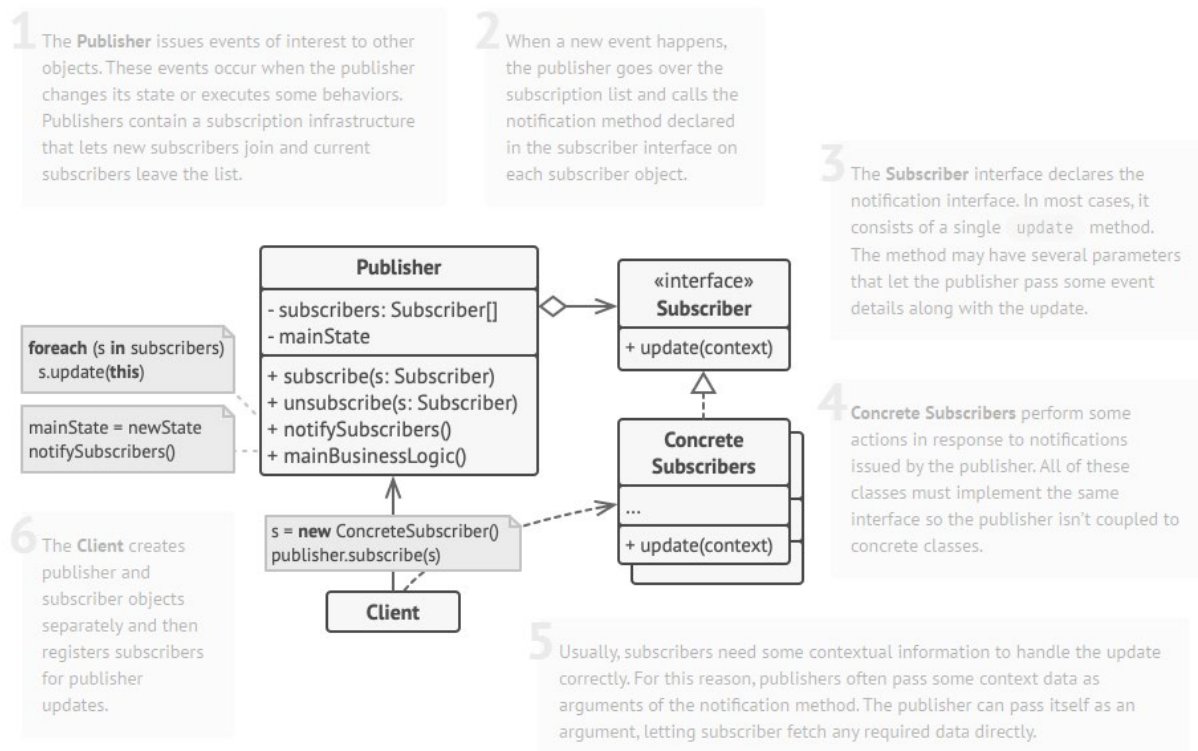
Forces. Minimize coupling between sender and receivers.

Solution. Subjects maintain subscriber lists and notify them on state change.

Consequences. Supports dynamic subscription; can cause update cascades.

Related. Mediator, Event Bus.

Uses. UI bindings, message brokers.



Proxy

(4-BSc_SWA_Conditions_Significance-Solution)

Problem. Access to an object must be controlled or extended transparently.

Context. Security, caching, or remote access.

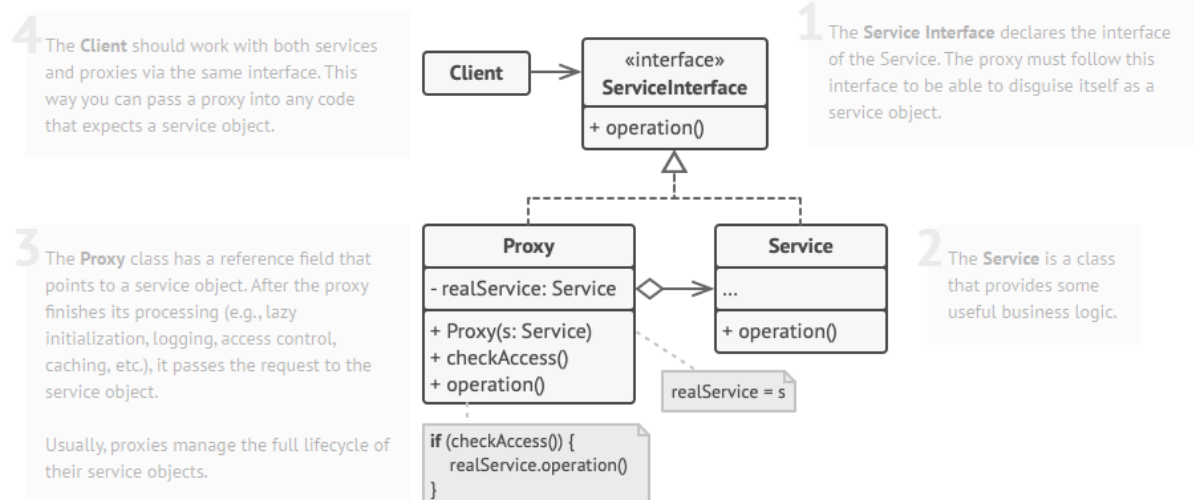
Forces. Maintain identical interface for client use.

Solution. Provide a proxy implementing the subject interface that adds behaviour before/after delegating.

Consequences. Adds flexibility and interception; small runtime cost.

Related. Decorator, Adapter.

Uses. Virtual proxies, RMI stubs, AOP interceptors.



Context Object

(4-BSc_SWA_Conditions_Significance-Solution)

Problem. Many collaborating components require shared environment data.

Context. Frameworks handling sessions, locales, or configurations.

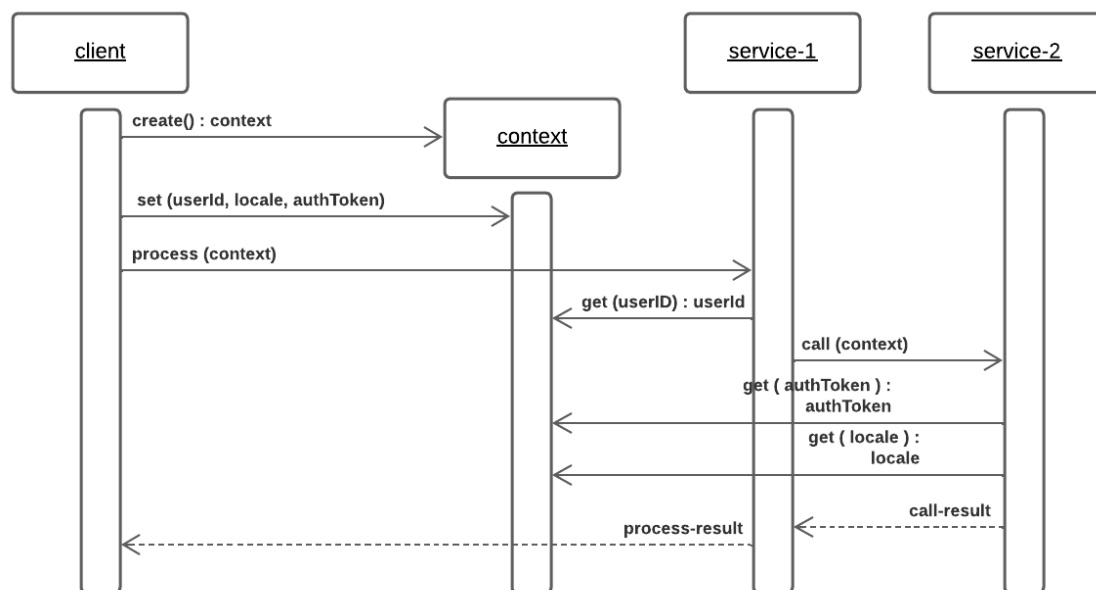
Forces. Avoid global variables yet share context consistently.

Solution. Encapsulate contextual information in a single object passed explicitly.

Consequences. Improves testability; requires explicit propagation.

Related. Dependency Injection, Service Locator.

Uses. ServletContext, transaction contexts.



Reflection

(4-BSc_SWA_Conditions_Significance-Solution)

Problem. A system must inspect or modify its own structure at runtime.

Context. Plug-in frameworks, scripting, dynamic binding.

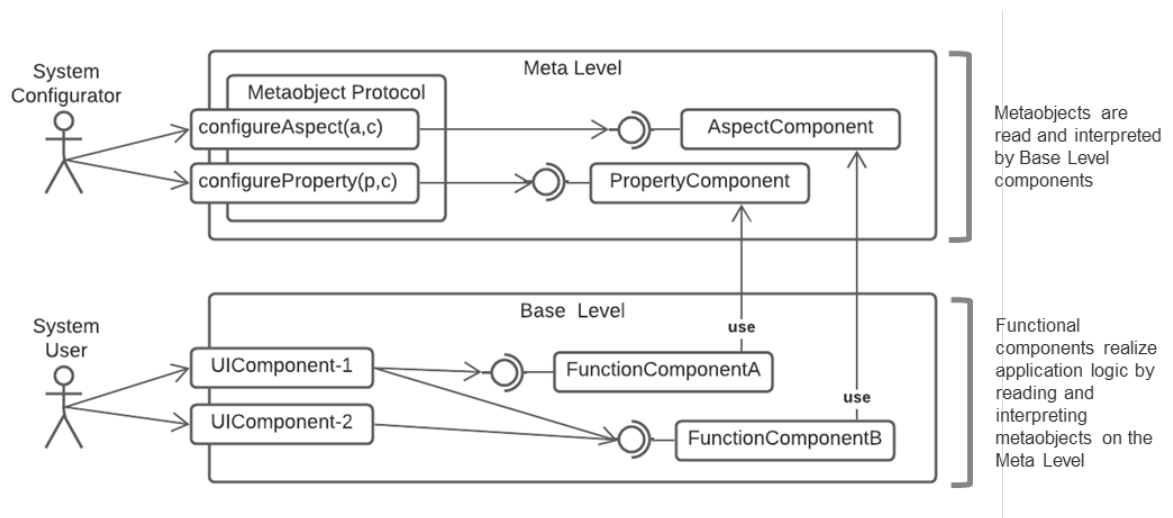
Forces. Balance flexibility and type safety.

Solution. Represent classes and methods as meta-objects manipulable at runtime.

Consequences. Enables dynamic adaptation; reduces performance.

Related. Microkernel, Interpreter.

Uses. Java Reflection API, dependency injection.



Chain of Responsibility

(6.1-BSc_SWA_WITH_WHAT_ArchitectureMeans_Overview)

Problem. Multiple handlers may process a request without the sender knowing who will handle it.

Context. Event processing, UI handling, middleware pipelines.

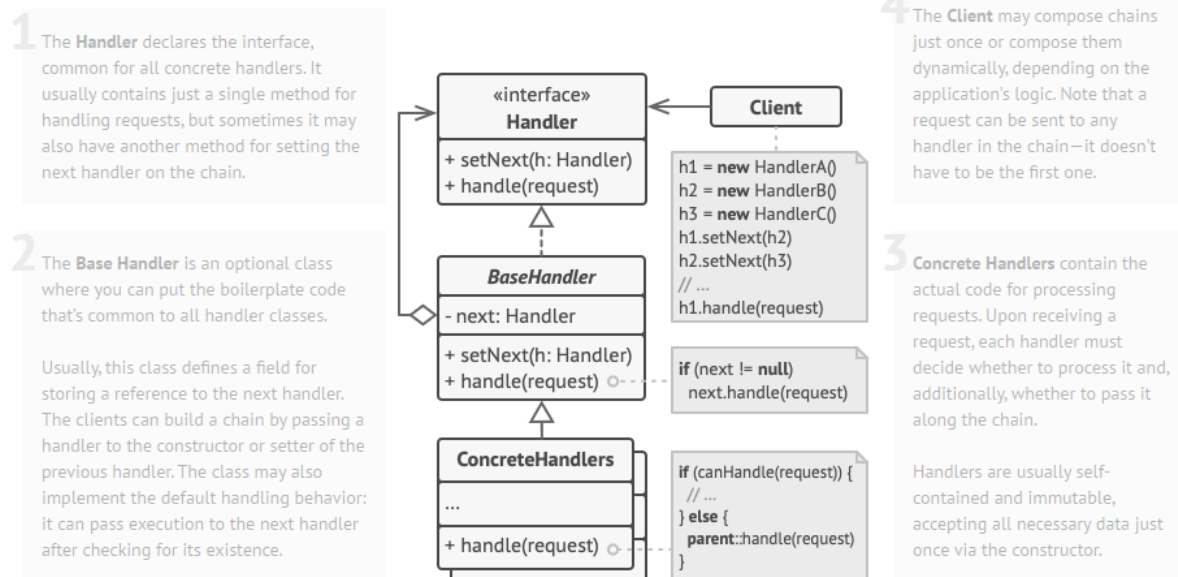
Forces. Decouple senders from specific receivers.

Solution. Link handlers into a chain; each processes or passes the request on.

Consequences. Flexible routing; harder debugging.

Related. Command, Decorator.

Uses. Servlet filters, exception chains.



Composite

(6.1-BSc_SWA_WITH_WHAT_ArchitectureMeans_Overview)

Problem. Clients must treat individual and composite objects uniformly.

Context. Hierarchical structures such as GUIs or file systems.

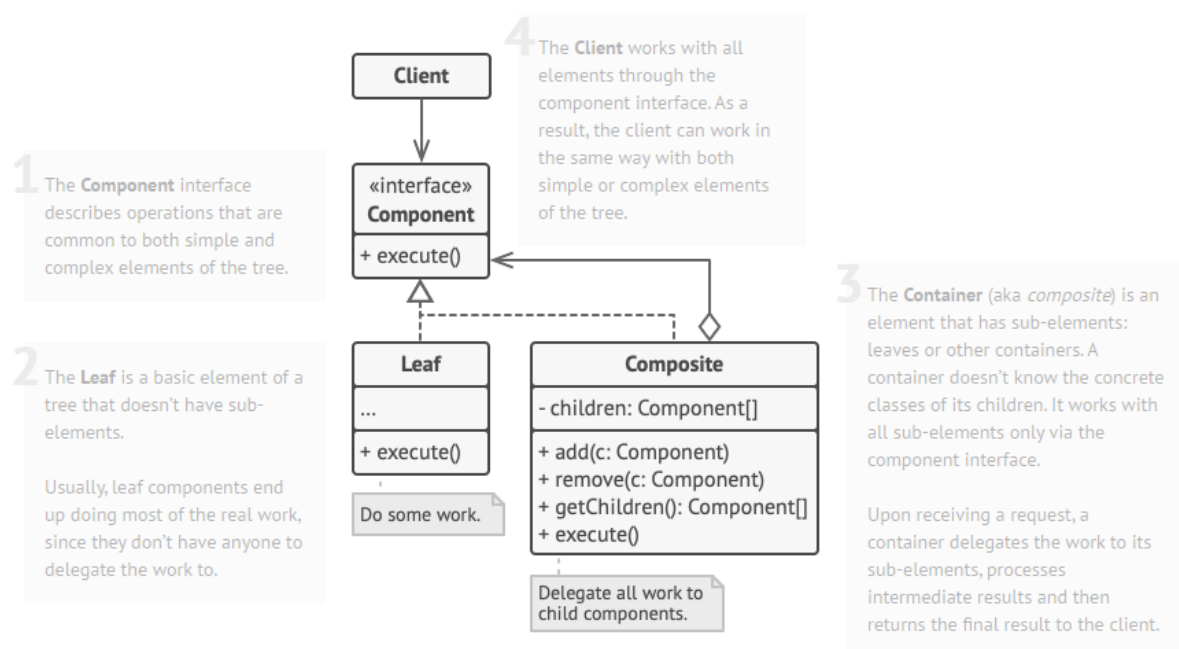
Forces. Uniform interface for parts and wholes.

Solution. Define a common component interface; composites forward operations to children.

Consequences. Simplifies recursion; can make constraints hard to enforce.

Related. Iterator, Decorator.

Uses. DOM trees, UI hierarchies.



Interpreter

(4-BSc_SWA_Conditions_Significance-Solution)

Problem. A language or grammar must be parsed and executed easily.

Context. Simple DSLs or configuration expressions.

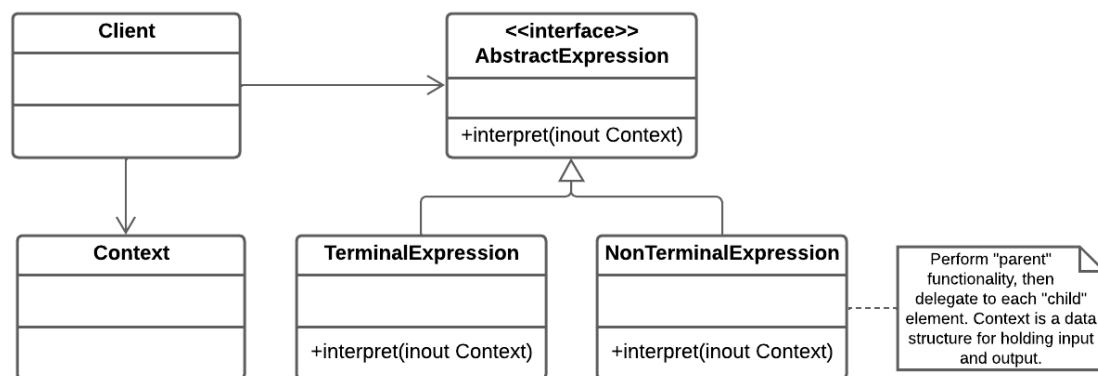
Forces. Flexibility vs. performance.

Solution. Represent grammar rules as class hierarchies; each node interprets itself.

Consequences. Easy extension; inefficient for large grammars.

Related. Composite, Visitor.

Uses. Expression evaluators, query filters.



State

(6-BSc_SWA_Method_DomainDrivenDesign-Solution)

Problem. Object behaviour changes with internal state.

Context. Workflow engines, game objects.

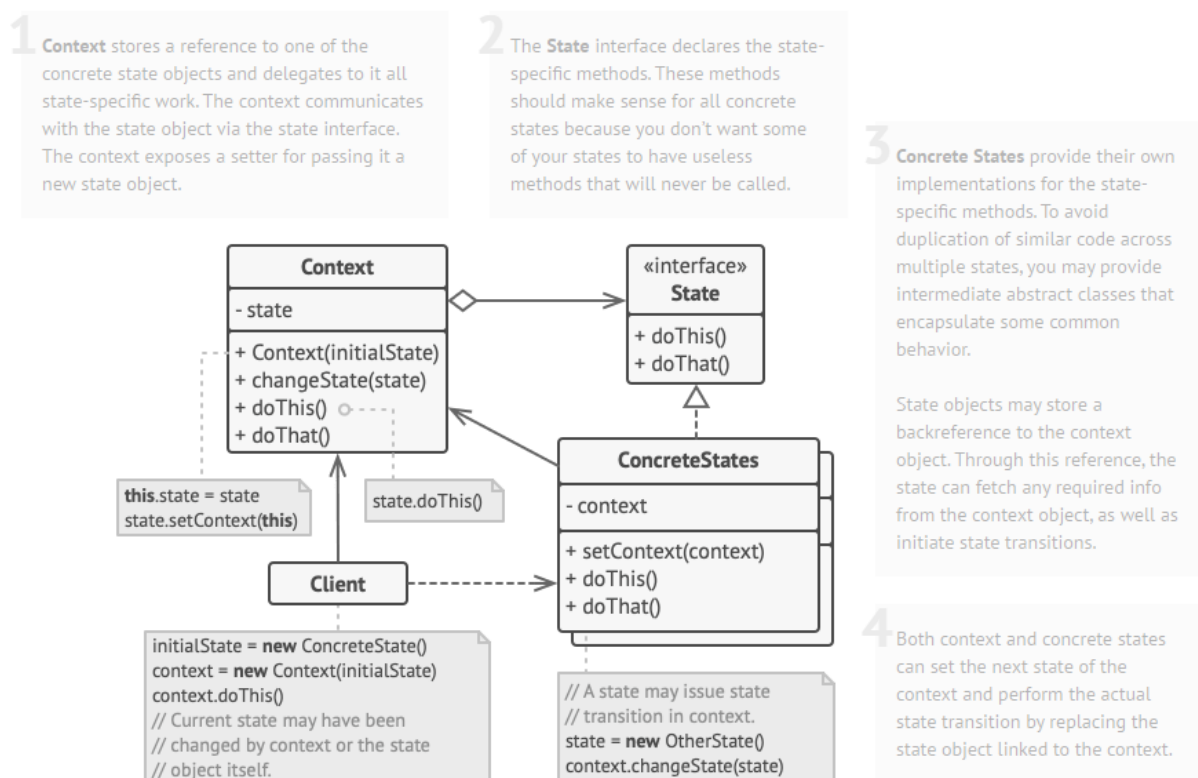
Forces. Avoid complex conditional logic.

Solution. Represent each state by a separate class sharing a common interface; context delegates behaviour.

Consequences. Simplifies transitions; increases object count.

Related. Strategy, Flyweight.

Uses. TCP connection states, vending machines.



Template Method

(6-BSc_SWA_Method_DomainDrivenDesign-Solution, 6.2-BSc_SWA_WITH_WHAT_ArchitecturePrinciples)

Problem. Parts of an algorithm vary, but its overall structure is fixed.

Context. Frameworks defining processing skeletons.

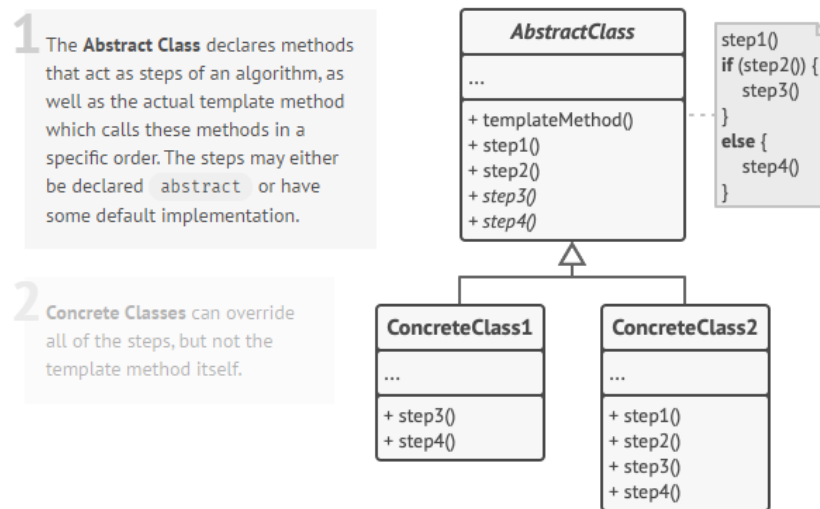
Forces. Share algorithm core, allow controlled customization.

Solution. Define a template method calling abstract steps implemented by subclasses.

Consequences. Promotes reuse; limits flexibility at runtime.

Related. Strategy, Factory Method.

Uses. Parsing frameworks, batch job templates.



Decorator

(6.2-BSc_SWA_WITH_WHAT_ArchitecturePrinciples)

Problem. Functionality must be extended dynamically without subclassing.

Context. Systems requiring runtime behaviour extension.

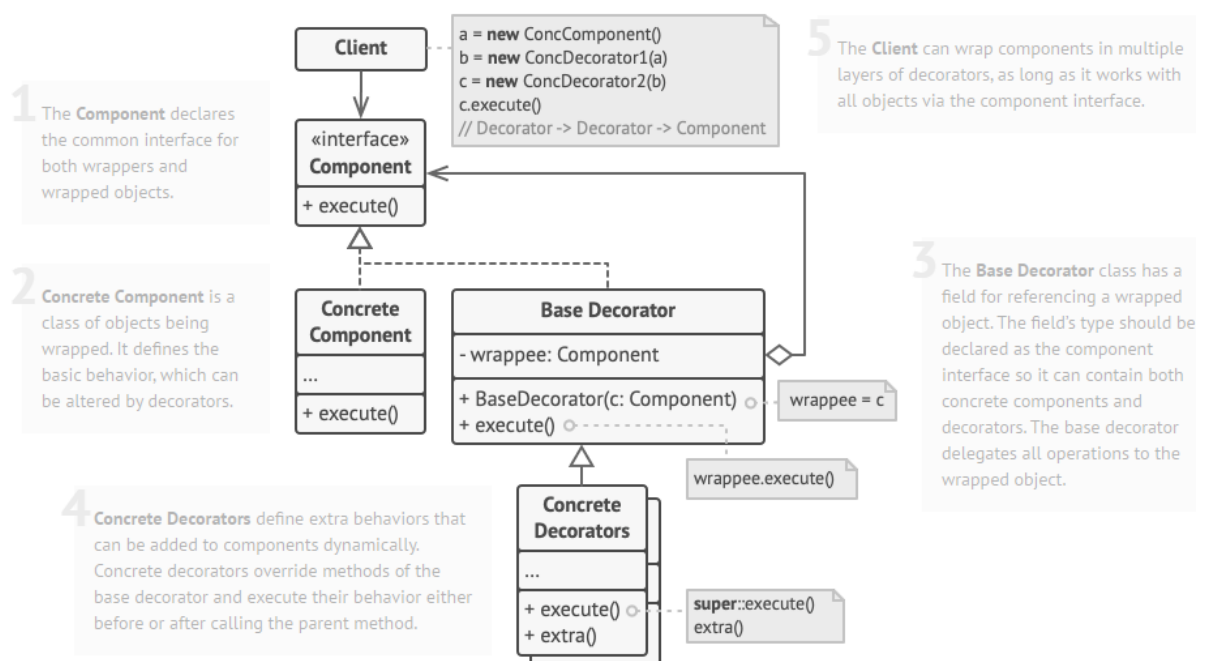
Forces. Preserve original interface and composability.

Solution. Wrap components in decorators implementing the same interface and add pre/post actions.

Consequences. Flexible layering; many small objects.

Related. Proxy, Composite.

Uses. Stream I/O filters, GUI features.



Domain Model

(8.1-BSc_SWA_WITH_WHAT_ArchitecturePatterns-MudToStructure, 8-BSc_SWA_Means_ArchitecturePatterns-Solution)

Problem. Business logic scattered across technical layers becomes inconsistent.

Context. Enterprise applications with complex business rules.

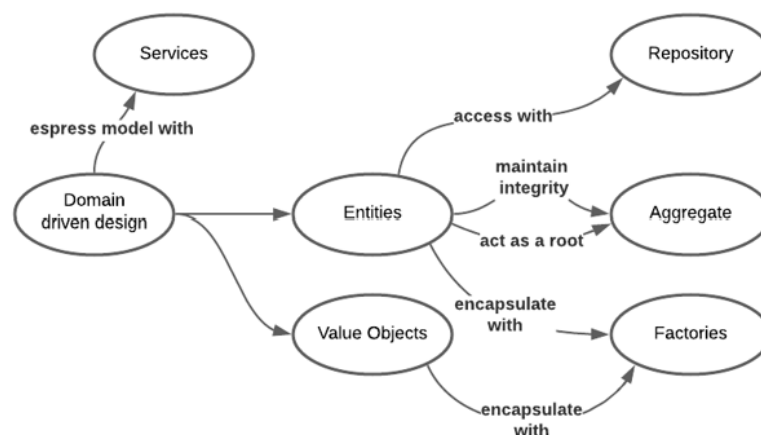
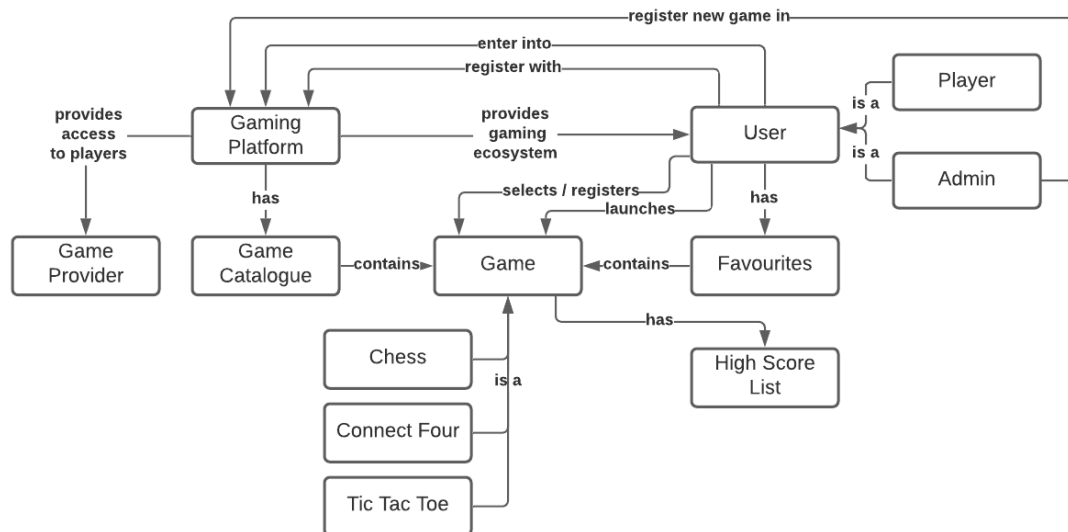
Forces. Maintain expressiveness and separation from infrastructure.

Solution. Model the core domain explicitly as interrelated objects with behaviour.

Consequences. High clarity and testability; requires mapping to persistence.

Related. Domain Object, Repository.

Uses. DDD architectures, ERP cores.



Domain Object

(8.1-BSc_SWA_WITH_WHAT_ArchitecturePatterns-MudToStructure, 8-BSc_SWA_Means_ArchitecturePatterns-Solution)

Problem. Business concepts need coherent data and behaviour representation.

Context. Object-oriented business systems.

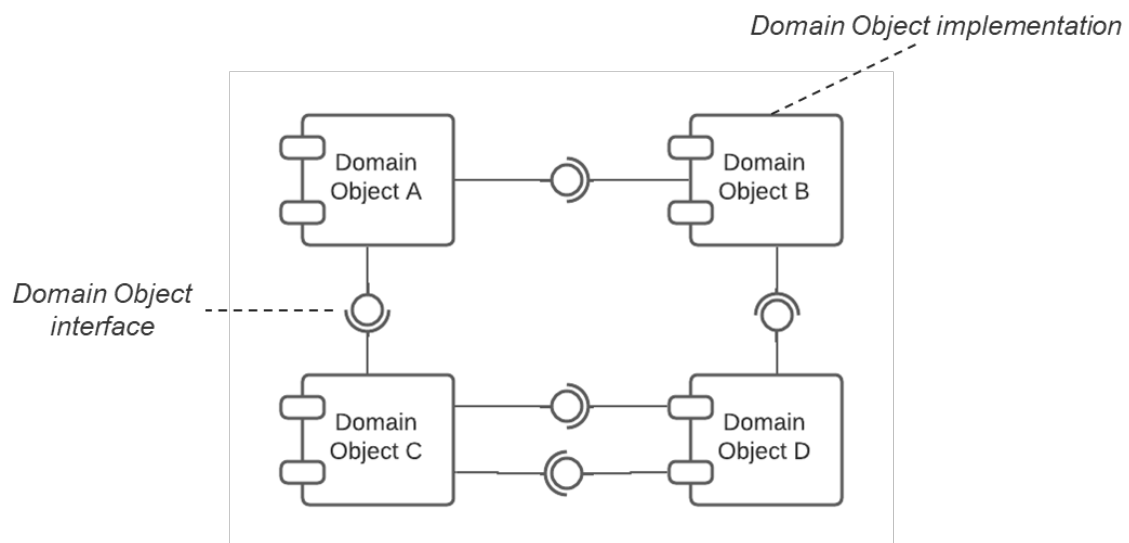
Forces. Keep logic close to data.

Solution. Represent each business entity as a class with attributes and methods.

Consequences. Improves cohesion; may duplicate DB schema.

Related. Entity, Value Object, Aggregate.

Uses. Order, Invoice, Customer entities.



Layers

(8.1-BSc_SWA_WITH_WHAT_ArchitecturePatterns-MudToStructure, 8-BSc_SWA_Means_ArchitecturePatterns-Solution)

Problem. Mixing presentation, business, and technical logic causes rigidity.

Context. Large systems with clear concern boundaries.

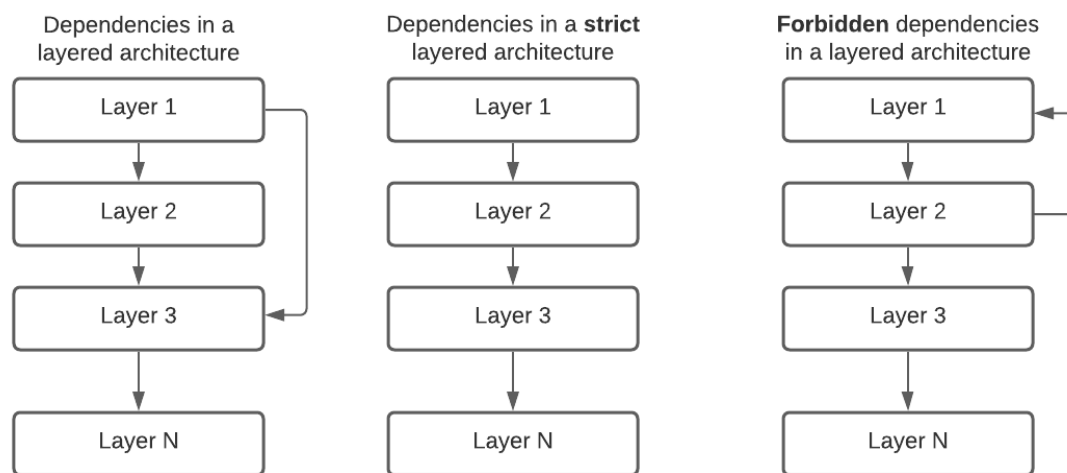
Forces. Separation vs. performance overhead.

Solution. Structure the system in stacked layers; each uses services of the one below through interfaces.

Consequences. Enhances modifiability; may slow communication.

Related. MVC, Microkernel.

Uses. 3-tier web apps, OSI stack.



Model-View-Controller

(8.1-BSc_SWA_WITH_WHAT_ArchitecturePatterns-MudToStructure, 8-BSc_SWA_Means_ArchitecturePatterns-Solution)

Problem. Tight coupling between UI and business logic limits reuse.

Context. Interactive applications needing multiple views.

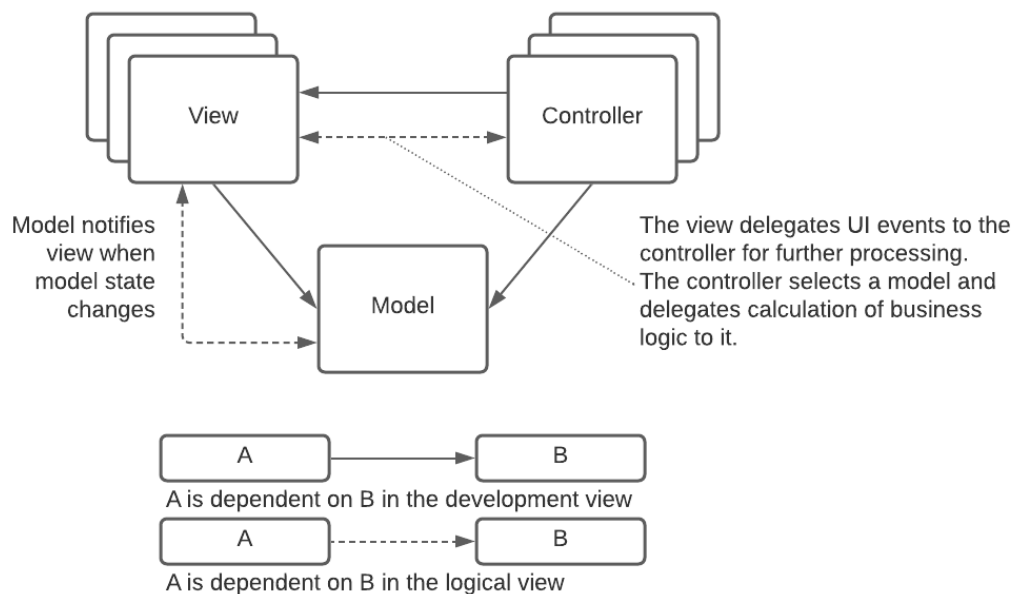
Forces. Separate concerns yet synchronize state.

Solution. Divide roles into Model (data), View (presentation), and Controller (input coordination).

Consequences. Improves flexibility; adds communication complexity.

Related. Observer, Layers.

Uses. Web frameworks, GUI toolkits.



Pipes and Filters

(8.1-BSc_SWA_WITH_WHAT_ArchitecturePatterns-MudToStructure, 8-BSc_SWA_Means_ArchitecturePatterns-Solution)

Problem. Data must undergo a series of independent transformations.

Context. Streaming or batch data processing.

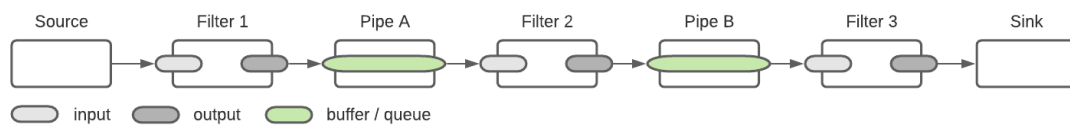
Forces. Need for composable, reusable steps.

Solution. Structure system as filters connected by pipes passing data streams.

Consequences. Encourages reuse and parallelism; can increase latency.

Related. Chain of Responsibility, Layers.

Uses. UNIX pipelines, ETL systems.



Shared Repository

(8.1-BSc_SWA_WITH_WHAT_ArchitecturePatterns-MudToStructure, 8-BSc_SWA_Means_ArchitecturePatterns-Solution)

Problem. Multiple components require shared access to consistent data.

Context. Systems emphasizing data integrity and coordination.

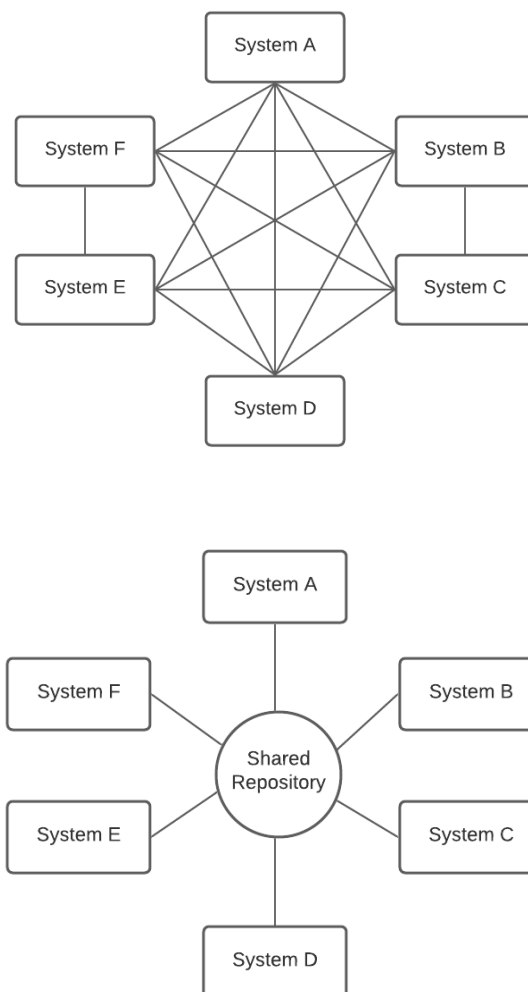
Forces. Centralized data vs. component autonomy.

Solution. Maintain a central data store accessed via defined interfaces.

Consequences. Simplifies consistency; introduces single-point dependency.

Related. Blackboard, Database, Layers.

Uses. Compilers, data hubs, version control systems.



Microkernel

(8.1-BSc_SWA_WITH_WHAT_ArchitecturePatterns-MudToStructure, 8-BSc_SWA_Means_ArchitecturePatterns-Solution)

Problem. Systems must be easily extensible without affecting a stable core.

Context. Product lines, plug-in platforms, OS kernels.

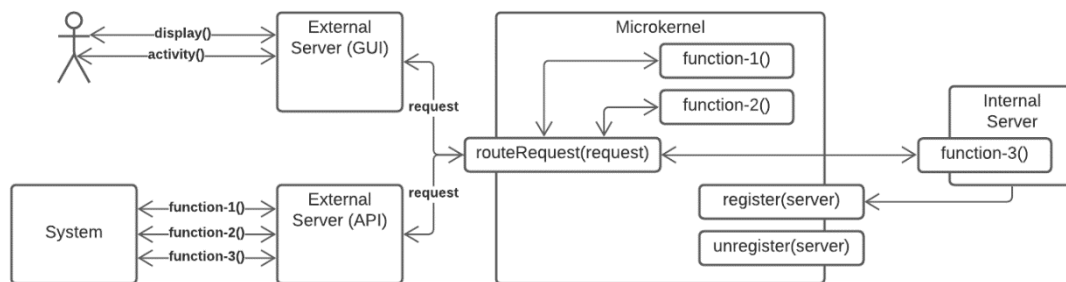
Forces. Core stability vs. extension flexibility.

Solution. Define a minimal core offering basic services and extension points; plug-ins add functionality.

Consequences. Enables portability and modularity; adds indirection.

Related. Reflection, Layers.

Uses. Eclipse, JBoss, Mach kernel.



Abstract Factory Class

Problem. Classes depend on families of related objects that must vary together.

Context. Systems supporting multiple configurations or platforms.

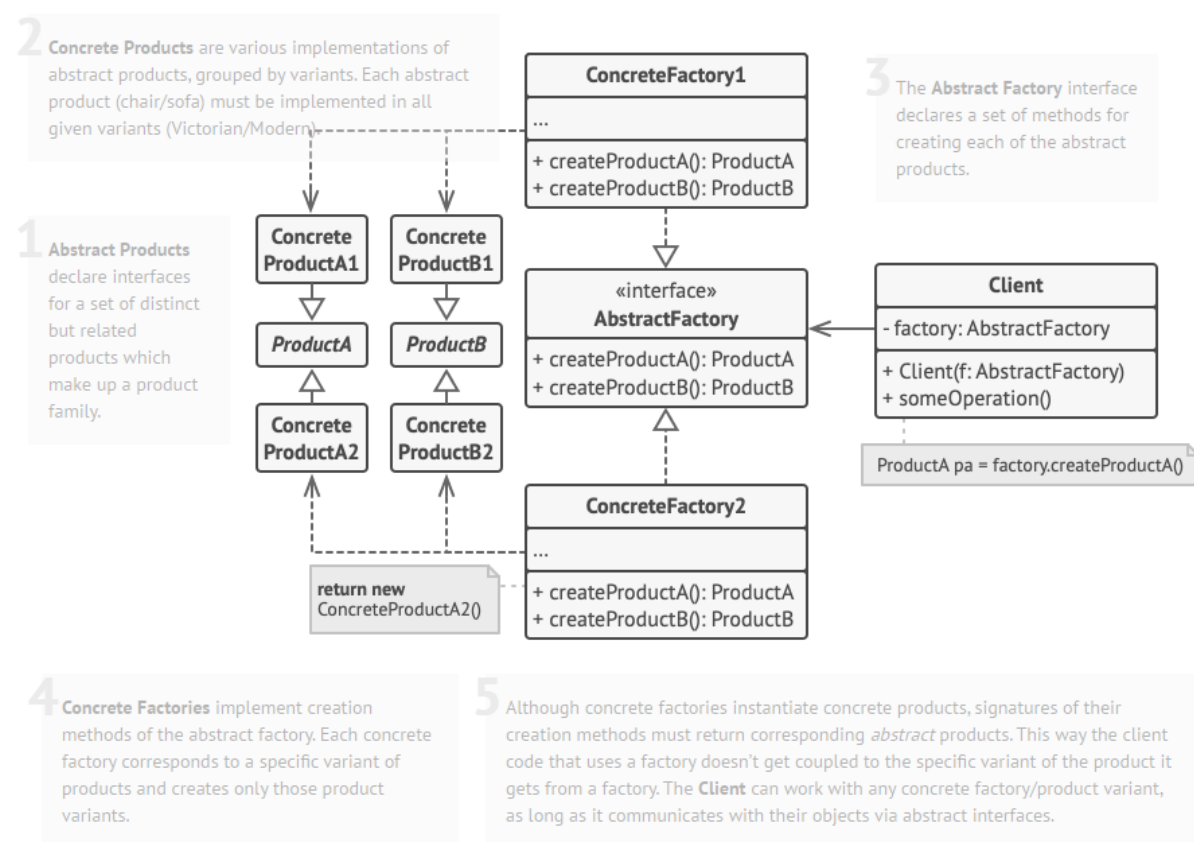
Forces. Encapsulate creation logic and keep code independent of concrete types.

Solution. Define an interface for creating families of objects; implement concrete factories per variant.

Consequences. Promotes consistency; adds abstraction layer.

Related. Factory Method, Prototype.

Uses. GUI toolkits (Windows vs. Mac), data provider APIs.



Mediator

Problem. Many-to-many communication between components creates tight coupling.

Context. Collaborative UI widgets, chat systems.

Forces. Reduce dependencies while preserving coordination.

Solution. Introduce a mediator that centralizes interactions among components.

Consequences. Simplifies component logic; mediator can grow complex.

Related. Observer, Facade.

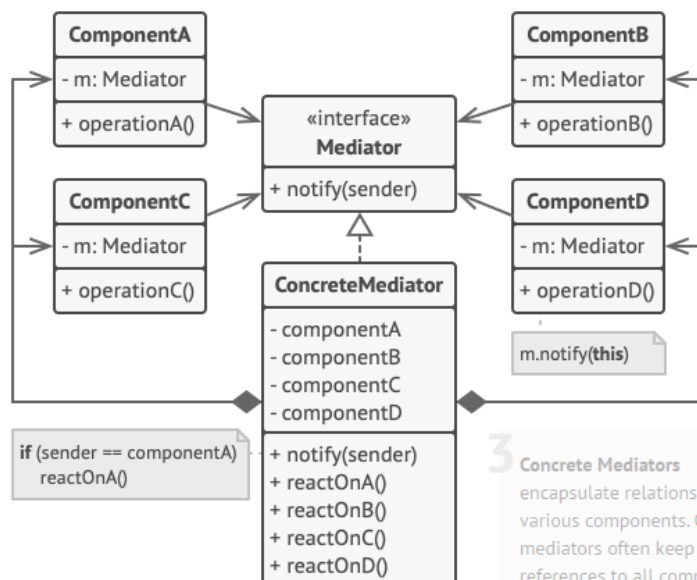
Uses. Dialog boxes, air-traffic-control simulations.

1 **Components** are various classes that contain some business logic. Each component has a reference to a mediator, declared with the type of the mediator interface. The component isn't aware of the actual class of the mediator, so you can reuse the component in other programs by linking it to a different mediator.

4 Components must not be aware of other components. If something important happens within or to a component, it must only notify the mediator. When the mediator receives the notification, it can easily identify the sender, which might be just enough to decide what component should be triggered in return.

From a component's perspective, it all looks like a total black box. The sender doesn't know who'll end up handling its request, and the receiver doesn't know who sent the request in the first place.

2 The **Mediator** interface declares methods of communication with components, which usually include just a single notification method. Components may pass any context as arguments of this method, including their own objects, but only in such a way that no coupling occurs between a receiving component and the sender's class.



3 **Concrete Mediators** encapsulate relations between various components. Concrete mediators often keep references to all components they manage and sometimes even manage their lifecycle.